

Constructing a Constant-Scaling Density Estimator

Sarah You

Department of Statistics and Actuarial Science
University of Western Ontario
London, Canada
yyou46@uwo.ca

Abstract—I will definitely, absolutely, finish this one day
Index Terms—Beta Distribution

INTRODUCTION

The current most popular estimation methods of unknown underlying probability density function from an observed sample of data is to apply kernel density estimation. The construction is as follows: Let $(x_1, x_2, \dots, x_n)$ be independent and identically distributed samples drawn from some univariate distribution with unknown density f at any given point x_i . To estimate the shape of function f , we take a moving average estimate, called a *kernel*, shown below,

$$\hat{f}_\lambda(x) = \frac{1}{n} \sum_{i=1}^n K_\lambda(x - x_i) = \frac{1}{n\lambda} \sum_{i=1}^n K\left(\frac{x - x_i}{\lambda}\right)$$

where K is the smoothing kernel and λ is a smoothing parameter called the bandwidth ([1]). The kernel function K must obey several properties. It must be a smooth function where $\int K(x)dx = 1$, $\int xK(x)dx = 0$, and $\sigma_K^2 = \int x^2 K(x)dx > 0$ ([2]). This technique provides a smooth estimate of the pdf and uses all sample points, which is not possible to do by looking at the histogram alone, the once popular method of estimating a sample's underlying distribution ([3]).

However, there are several disadvantages to performing Kernel Density Estimation. Firstly, kernel density estimation produces a constant bias, particularly near the boundaries of datasets with a bounded support. Furthermore, if the underlying distribution has a longer tail/tails, the "main" component of the distribution risks over-smoothing ([4]). Considering the Beta distribution both has a bounded support over $(0, 1)$, and has heavy-tailed distributions depending on the magnitude of their shape and scale parameters, Kernel Density Estimation is wholly not suitable. This article attempts to find a method to use constant scaling to estimate if an underlying distribution is a beta distribution, which could significantly simplify the estimation process both in time and computational resources.

METHODOLOGY

A. Simulation Procedure for Trend Estimation

Data for the following analysis were simulated using the following procedure.

- 1) Let $X_1, X_2, \dots, X_n \sim \text{Beta}(\alpha, \beta)$, $n \in \mathbb{N}$ and is sufficiently large, and $\alpha, \beta > 0$.

- 2) Let $Y_1, Y_2, \dots, Y_n$ be the upper p percent of $X_1, X_2, \dots, X_n$. Define

$$\bar{y} = \frac{\sum_{i=1}^n y_i}{n}$$

and

$$s^2 = \frac{\sum_{i=1}^n (y_i - \bar{y})^2}{n - 1}$$

- 3) Construct a constant c such that $cs + Y_{(n)} = 1$, where $Y_{(n)}$ denotes the sample maximum. Then

$$c = \frac{1 - Y_{(n)}}{s}, \quad c > 0$$

- 4) Repeat steps 1 to 3 for $k \in \mathbb{N}$ samples of the same size n to obtain $c_1, c_2, \dots, c_k > 0$. Then the average of the constants

$$\bar{c} = \frac{\sum_{i=1}^k c_i}{k}$$

- 5) Repeat steps 1 to 4 for different sample sizes $N_1, N_2, \dots, N_j$, $j \in \mathbb{N}$. Then the average c^* of sample size N_j is

$$c^* = \frac{\sum_{i=1}^j \bar{c}}{j}$$

- 6) Repeat steps 1 to 5 to obtain $c_1^*, c_2^*, \dots, c_m^*$ with different $\text{Beta}(\alpha_p, \beta_p)$ for some $p \in \mathbb{N}$ and $\forall \alpha_p, \beta_p > 0$.

The procedure is ran using Python NumPy using the below pseudocode (Algorithm 1). For ease of analysis of the Beta distribution is restricted to $\alpha \in [2, 10]$ and $\beta \in [\frac{1}{2}, 10]$, a standard range of shape and scale parameter values for some proportion p . Additionally 10 random samples of size 200, 500, 1000, 5000 and 10,000 were used. The function `get-cs`, which covers steps 1-3 of the procedure, is clarified in Algorithm 2. Preliminary findings of the simulation results are discussed in subsection -B.

Algorithm 1 Algorithm for simulating data points

Require: $\alpha \geq 0, \beta \geq 0, p \in (0, 1)$ **Ensure:** $c^* > 0$

```
 $p \leftarrow \text{input}$ 
for all  $\alpha \leftarrow [2, 10]$  and  $\beta \leftarrow [2, 10]$  do
  for all  $i \leftarrow [200, 500, 1000, 5000, 10000]$  do
    count  $\leftarrow 1$ 
    repeat
       $c \leftarrow$ 
      function GET-CS( $\alpha, \beta, p$ )
    until count  $\geq i$ 
     $\bar{c} \leftarrow$ 
    function AVERAGE( $c$ )
   $c^* \leftarrow$ 
  function AVERAGE( $\bar{c}$ )
```

Algorithm 2 Algorithm for GET-CS

Require: $\alpha \geq 0, \beta \geq 0, p \in (0, 1), i$ **Ensure:** $c > 0, \bar{c} > 0, c^* > 0$

```
 $p \leftarrow \text{input}$ 
points  $\leftarrow$ 
function SIMULATE-BETA-DISTRIBUTION( $\alpha, \beta$ )
if points  $\geq$ 
  function PERCENTILE(points,  $p$ )
  then
    sample  $\leftarrow$ 
    function GET-POINTS(points,  $p$ )
sample-max  $\leftarrow$ 
function GET-MAX(sample)
sd  $\leftarrow$ 
function GET-STANDARD-DEVIATION(sample)
 $c \leftarrow$ 
function GET-C(sample-max, sd)
return  $c$ 
```

B. Basic Findings and Graphic Output



During the simulation process an inverse trend between c and the simulation size N is observed, with c decreasing to 0 as N increases for a fixed $Beta(\alpha, \beta)$ distribution. Additionally, s^2 for each sample is fairly low as all simulated points lie in the support of $(0, 1)$ and on top of that only the top p percent are taken; this leads to the value of c being dependent on the sample maximum $Y_{(n)}$, which is in turn dependent on α and β .

The particular relationship is more obvious upon observation of the following graphic output (Figure 1). The relationship between α and β is the biggest determinant of the magnitude of c . Compared to the base scenario of $\alpha = \beta$, which produces a symmetric distribution and a moderate c value, having $\alpha < \beta$ creates a right skewed distribution and thus c is larger as $Y_{(n)}$ tends to be smaller. Conversely, when $\alpha > \beta$, the distribution skews left, producing a larger $Y_{(n)}$ in most cases, which leads to smaller c . Furthermore, the bigger

$|\alpha - \beta|$ is, the more extreme c becomes compared to c for $Beta(\alpha, \beta)$.

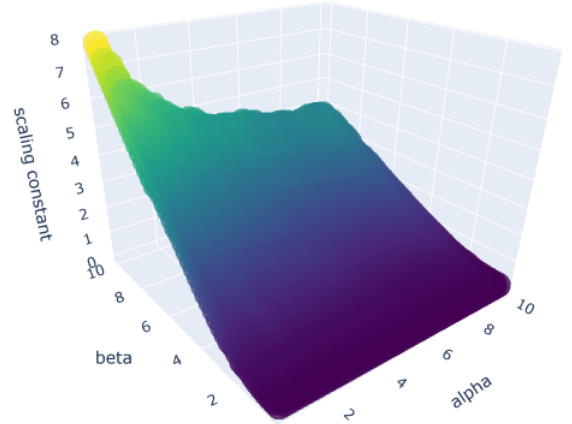


Fig. 1. Graphical representation of c values produced by procedure defined in Algorithm 1.

C. Ongoing Difficulties

So far I have attempted the following methods to try and adjust c^* so c^* is always constant.

- Currently p is set to the 66.7th percentile (ie. the top 1/3rd of simulated points are taken for the procedure). Per our conversations I have tried $p = 75, 80, 85, 90, 92, 95$ and 99. Similarly to the previous situation, there is no visible change in the distribution of c s. (See figure 2)
- Based on a previous attempt, I tried adjusting p by the skew of the distribution when sampling for the upper portion of data. I was hoping with p now varying depending on sample skew; for positively skewed (ie. right skewed) Beta distributions a smaller proportion of points are chosen, thus hopefully decreasing s and increasing c . The logic for negatively skewed distributions apply vice versa. Methods of adjusting p included adding p by $a \times \text{skew}$ for any $a \in \mathbb{R}$, and multiplying by a^{skew} , where $a \in (1, \frac{100}{p})$. (See figure 3) This time however, there was no noticeable change in c compared to the basic procedure.
- Based on the roughly exponential shape of the graph I adjusted my calculation of c^* . I have tried $c^* = e^{-(\sum_{i=1}^j \bar{c})/j}$, keeping c^* the same but letting $\bar{c} = e^{-(\sum_{i=1}^k c_i)/k}$, and letting both c^* and $\bar{c}$ stay constant but changing c to $e^{-(1-Y_{(n)})/s}$. In all three cases the resulting c values exhibit an even more extreme exponential trend. (See figure 4)
- Since adjusting by the magnitude of e^{-x} was too extreme, I tried to model $c' = \frac{1}{c}$ which yielded the same graphical results as above. Although the graph looks mostly flat (see 5), the values increase drastically as α and β decrease,



up to 5×10^8 . A similar graphic result occurs when I try to calculate $c = (1 - Y_{(n)})/(s(\alpha + \beta))$. If I divide c by α or β alone, there is no change in the distribution of c . (see figure 6)

As you can see none of the methods work. Currently by adjusting c to

$$c' = \frac{[1 - Y_{(n)} \times \frac{skew}{1000}]}{s}$$

I produce a $f(c)$ that is closer to constant, but still very flawed (See figure 7). I'm not sure what to do, please help.

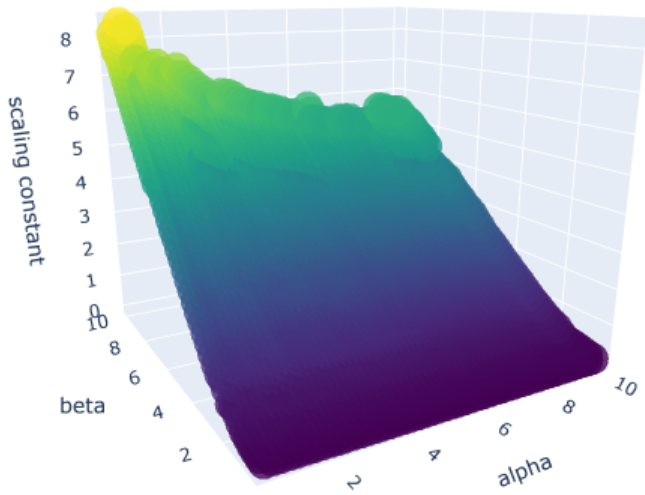


Fig. 2. Graph of $f(c)$ with $p = 90$.

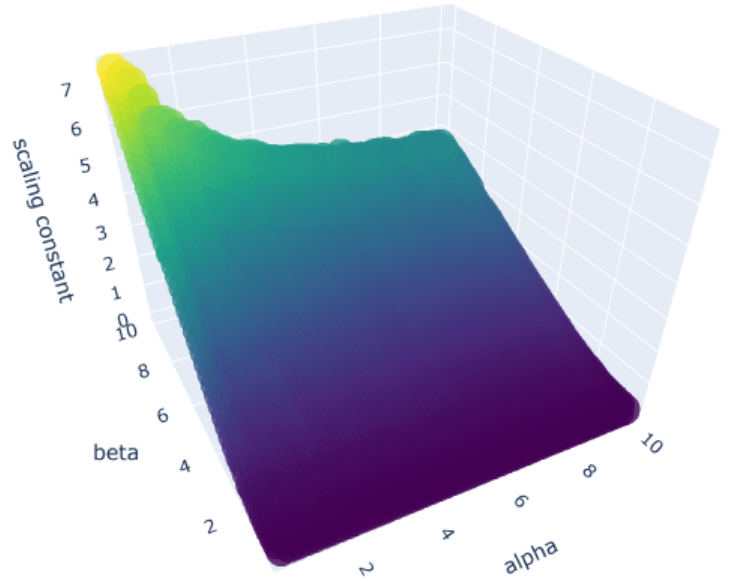


Fig. 3. Graph of $f(c)$ with p th percentile adjusted by skewness.

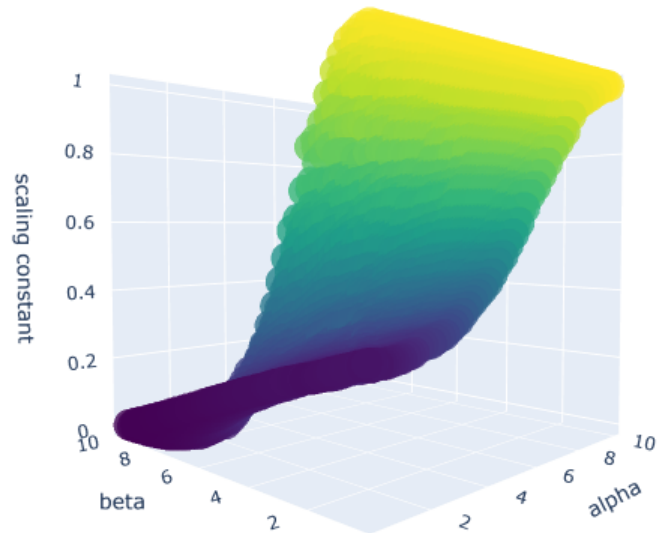


Fig. 4. Graph of $f(c)$ where $c = e^{-(1-Y_{(n)})/s}$. The other exponent methods produced similar results.

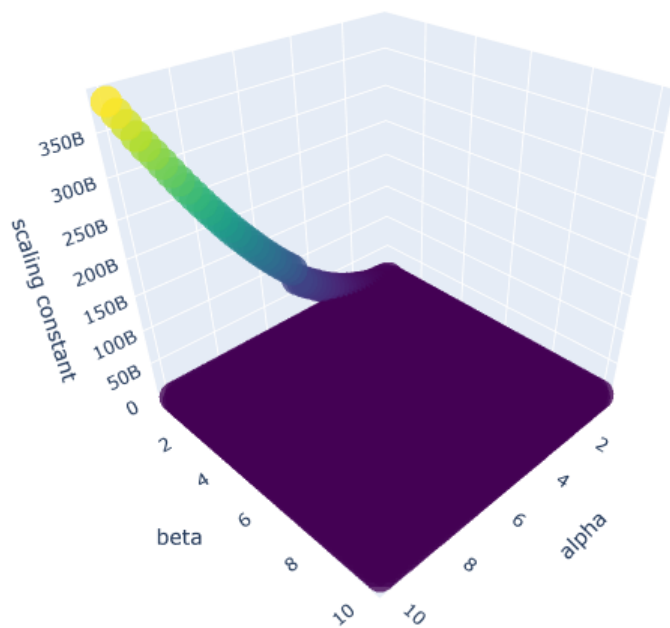


Fig. 5. Graph of $f(c)$ where $c' = 1/c$. $c' = c/(\alpha + \beta)$ produced similar results.

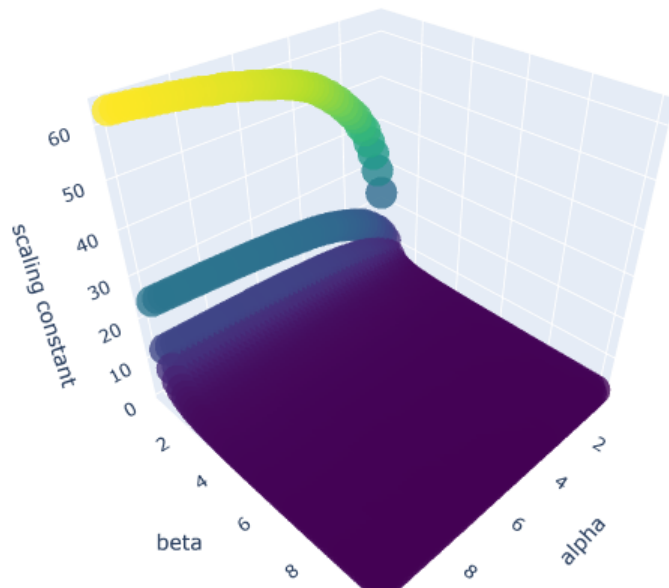


Fig. 7. Graph of best attempt at making $f(c)$ constant.

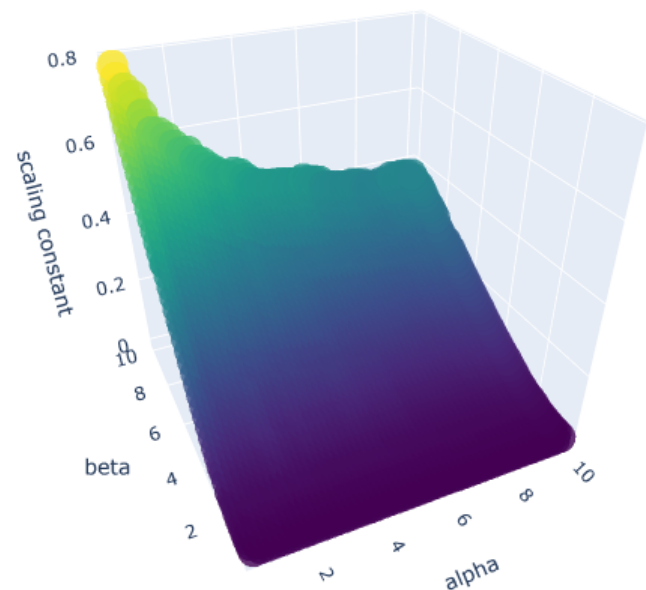


Fig. 6. Graph of $f(c)$ where $c' = c/\alpha$ or c/β .

REFERENCES

- [1] J. J. Faraway, *Extending the Linear Model with R: Generalized Linear, Mixed Effects and Nonparametric Regression Models*, 2nd ed. University of Bath, UK: CRC Press, 2016, p. 299.
- [2] L. Wasserman, "Density estimation 36-708," class Lecture.
- [3] S. Węglarczyk, "Kernel density estimation and its application," in *ITM Web of Conferences*, vol. 23, Krakow, Poland, nov 2018, pp. 37–45.
- [4] A. Z. Zambom and R. Dias. (2012, dec) A review of kernel density estimation with applications to econometrics. arXiv. [Online]. Available: arxiv.org/pdf/1212.2812